# Android based Crowdsourced Disaster Alert & Safety App

## Software Requirements Specification

**Version 1.0**

**Group Id: F25PROJECT384C7**

**Supervisor Name : Irshad Nasir**

# Revision History

| Date (dd/mm/yyyy) | Version | Description | Author |
|---|---|---|---|
| 01-12-2025 | 1.0 | Natural disasters such as floods, earthquakes, and fires can strike unexpectedly, putting lives and property at risk. Timely information during such events is crucial, yet traditional communication channels are often slow or unreliable. The **Crowdsourced Disaster Alert & Safety App** is an Android-based mobile application designed to address this challenge. It enables users to report disasters in real time, receive instant alerts based on their location, and verify reports collectively. By integrating features like interactive maps, SOS alerts, and an admin verification system, the app leverages community participation and cloud technologies to enhance disaster preparedness, response, and overall public safety. | bc240222825 |

# Table of Contents

1. Scope (of the project)
2. Functional Requirements Non Functional requirements
3. Use Case Diagram
4. Usage Scenarios
5. Adopted Methodology
6. Work Plan (Use MS Project to create Schedule/Work Plan)

# SRS Document

## Scope of Project:

The scope of the **Crowdsourced Disaster Alert & Safety App** covers the end-to-end creation of an Android mobile application that empowers communities to report and respond to disasters in real time. This project focuses on building a reliable digital platform where users can instantly report incidents such as floods, fires, earthquakes, accidents, or any emergency they witness. Users will be able to provide essential details including the disaster type, a short description, their GPS-based location, and optional photo or video evidence. All submitted reports will be stored securely in Firebase using either Firestore or the Realtime Database, making the information available to everyone on the app almost immediately.

A major part of the project's scope involves developing a **map-based interface** that displays all reported incidents using interactive markers, helping users visually understand what is happening around them. The app will also include a **community verification system**, allowing users to confirm or dismiss reports. This crowdsourced validation helps minimize false alarms and ensures more reliable information for everyone.

To further enhance trust and accuracy, the project includes an **admin panel** where authorized personnel can review user submissions, verify reports, update their status, or mark them as resolved or false. Instant communication is also in scope—using Firebase Cloud Messaging—to alert users whenever a new disaster is reported near their location or when the status of their submitted report changes.

In addition to disaster reporting, the scope includes developing a **dedicated SOS feature** that allows users to send emergency alerts along with their live location to their saved contacts for immediate assistance. The app will also handle media uploads using Firebase Storage, implement secure login through Firebase Authentication, and present an intuitive, easy-to-use interface built in Android Studio using Java/Kotlin and XML.

The overall scope covers requirements gathering, UI/UX design, core feature development, testing, optimization, documentation, and the deployment of a complete Android APK. The project, however, does **not** include building an iOS version, offline-only usage, or support for third-party cloud platforms other than Firebase.

Functional and non Functional Requirements:

## Functional Requirements

### 1. User Registration & Login

The app should let users create an account easily using their email and password, and it should also support quick Google sign-in for convenience. Users must be able to log in securely every time, and if they forget their password, the app should let them recover it through a simple password-reset option. Once logged in, the session should stay active so users don't have to log in repeatedly unless they choose to sign out.

### 2. User Profile & SOS Contacts

Users should be able to manage their profile information and add the people they trust as emergency contacts. They should be able to add new contacts, edit their details, or remove them anytime. The app must make sure phone numbers are saved correctly and stored safely so they can be used during SOS alerts.

### 3. Creating Disaster Reports

Users should be able to report a disaster quickly by selecting the type of incident—like flood, fire, accident, or earthquake—writing a short description, and attaching photos or videos as proof. The app should automatically detect their GPS location, but users should still be able to edit

or adjust the pin on the map before submitting the report. Once submitted, the report should be saved in the cloud so it can be viewed by others immediately.

**4. Viewing Disaster Reports**

All reports should be visible in two ways: a simple list and an interactive map.

- The list view should show all active and previous reports along with brief details.

- The map view should display each report using a marker so users can instantly see what's happening nearby.
  Tapping any report should open its full details including the description, media, location, timestamp, and verification status.

**5. Community Verification**

Users should be able to help validate reports by confirming whether a reported event is true or dismissing it if they believe it's false. The app should automatically update the verification or dismissal counts and change the report's status based on how many users support it. This helps the community filter out false alarms.

**6. Notifications & Alerts**

When a new disaster report is submitted, especially one close to the user's current location—the app should immediately send a push notification.
Users should also receive alerts when:

- their own report gets verified,

- an admin updates the status, or

- a nearby disaster becomes more severe.

These notifications should be real-time and should help users stay informed.

**7. SOS/Emergency Feature**

The app should have a dedicated SOS button for emergencies. When pressed, it should instantly send the user's live location to all saved emergency contacts. The SOS alert should also be stored in the system so admins can monitor ongoing emergencies. The app should update the user's live location if they move while SOS is active.

**8. Admin Panel & Controls**

Admins should have access to a separate dashboard where they can view all submitted reports, verify their authenticity, update their status (verified, resolved, or false alarm), and remove inappropriate or misleading content. Admins should also be able to monitor SOS alerts and intervene if needed.

**9. Media Upload & Management**

Users should be able to upload disaster-related photos or videos while creating a report. These files should be stored safely in the cloud and appear clearly inside the report details. Admins should be able to remove any harmful, irrelevant, or inappropriate media if necessary.

**Non-Functional Requirements**

**1. Performance**

The app should feel smooth and responsive. Reports should load within a couple of seconds, and the map should display markers without any lag. Notifications should reach users almost instantly after a report is created or updated.

**2. Security**

All user information must be kept safe. Login details should be handled securely through Firebase Authentication, and the database should follow strict security rules so that only the right users can update or view sensitive

information. Location data, contacts, and media files must be protected from unauthorized access.

**3. Reliability**

The app should be available at all times, especially since emergencies can happen at any moment. If the user's internet connection becomes unstable, the app should alert them and automatically retry uploading the report or fetching updates once the connection is restored.

**4. Usability**

The app should be easy to use even for people with limited technical experience. The interface should be clean, with simple navigation and clear labels. Disaster types should be represented with intuitive icons and colors so users can recognize them instantly.

**5. Compatibility**

The app should run smoothly on Android devices starting from version 8.0 (Oreo) and work well on various screen sizes, including both smartphones and tablets.

**6. Scalability**

As the number of users grows, especially during widespread disasters, the app should be able to handle a large number of simultaneous reports, uploads, and notifications without slowing down. Firebase should scale automatically to support heavy traffic.

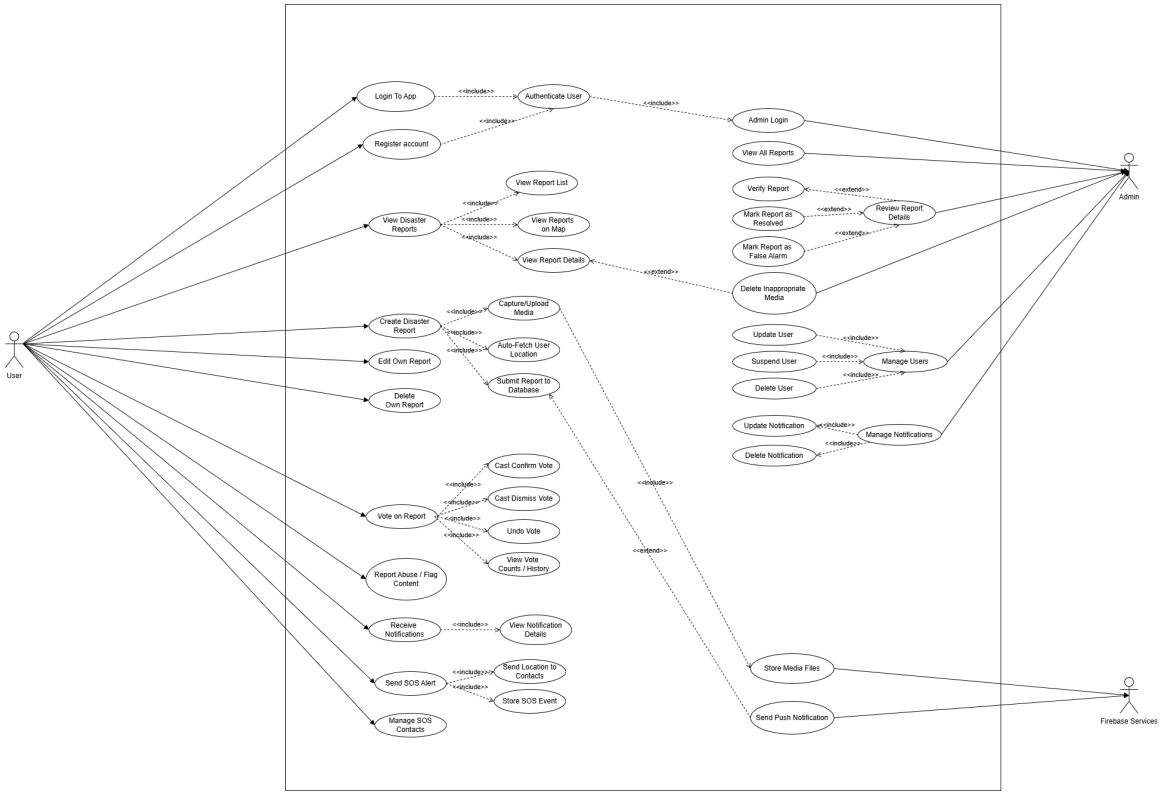
**7. Maintainability**

The code should be organized and well-documented so future developers can easily update or expand the app. Features like disaster categories, map settings, and notification options should be easy to modify if new requirements arise.

## 8. Privacy

Users' private data must be treated carefully. The app should only access the user's location after they approve it, and their emergency contacts and personal details should remain confidential. If a user decides to delete their account, the system should remove their personal data permanently.

<u>Use Case Diagram(s):</u>



Usage Scenarios:

| Field | Details |
|---|---|
| **Use Case Title** | Login to App |
| **Use Case ID** | UC-U01 |
| **Description** | This use case allows a registered user to securely access the application by entering their email and password. The system verifies these credentials through Firebase Authentication and redirects the user to the dashboard upon successful login. |
| **Actions** | 1. User opens the app.<br>2. User selects “Login”.<br>3. User enters email and password.<br>4. User taps the “Login” button.<br>5. System sends credentials to Firebase Authentication.<br>6. Firebase validates input and returns success or failure.<br>7. System loads user profile and opens home dashboard. |
| **Alternative Paths** | **A1:** User taps “Forgot Password” → Redirects to password recovery.<br>**A2:** User updates email/password before submitting.<br>**A3:** User chooses Google Sign-In (if available). |
| **Pre-Conditions** | User must already be registered and have active internet connectivity. |
| **Post-Conditions** | User session is created, user token is generated, and user is redirected to dashboard. |
| **Exceptions** | **E1:** Wrong password → System shows “Incorrect credentials”.<br>**E2:** Account suspended → System denies login.<br>**E3:** Network failure → System cannot contact Firebase.<br>**E4:** Invalid email format → Blocks submission. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | Register Account |
| **Use Case ID** | UC-U02 |
| **Description** | This use case enables new users to create an account using an email and password. The system validates input, ensures the email is not already in use, and stores the user in Firebase Authentication. |
| **Actions** | 1. User selects “Create Account”.<br>2. User enters required details (email, password, name).<br>3. User taps “Register”.<br>4. System validates email format and password strength.<br>5. Firebase checks if email already exists.<br>6. New user account is created.<br>7. User is automatically logged in and redirected to dashboard. |
| **Alternative Paths** | **A1:** Weak password → System prompts: “Password must include at least 6 characters.”<br>**A2:** Email already registered → Redirects user to Login page.<br>**A3:** User cancels registration. |
| **Pre-Conditions** | User must not already have an existing account with the same email. |
| **Post-Conditions** | A new user account is created and stored in Firebase Authentication. |
| **Exceptions** | **E1:** Invalid email format.<br>**E2:** Network problem prevents account creation.<br>**E3:** Firebase service outage.<br>**E4:** User closes the app mid-registration. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | Authenticate User |
| **Use Case ID** | UC-U03 |

| | |
|---|---|
| **Description** | The system verifies the user's identity through Firebase Authentication, ensuring access only to legitimate users. It manages token creation, token refresh, and secure session handling. |
| **Actions** | 1. User attempts login.<br>2. System sends credentials/token to Firebase Auth.<br>3. Firebase evaluates and returns success/failure.<br>4. System creates secure local session.<br>5. User gains access to app features. |
| **Alternative Paths** | **A1:** Expired token → System refreshes token automatically.<br>**A2:** User logs in using stored session token. |
| **Pre-Conditions** | Valid user account must exist. |
| **Post-Conditions** | User is authenticated and granted access to protected features. |
| **Exceptions** | **E1:** Token expired and refresh fails.<br>**E2:** Credentials invalid.<br>**E3:** Authentication server unreachable.<br>**E4:** Account disabled by admin. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | View Disaster Reports |
| **Use Case ID** | UC-U04 |
| **Description** | Allows users to browse all active and previous disaster reports submitted by the community. The system fetches structured data from Firestore and displays them in either list or map format. |
| **Actions** | 1. User opens home screen.<br>2. System retrieves all report data.<br>3. Reports displayed in card/list format.<br>4. User scrolls, filters, or selects a specific report. |

| | |
|---|---|
| **Alternative Paths** | **A1:** User applies filters (e.g., disaster type, severity, distance).<br>**A2:** User switches between list view or map view. |
| **Pre-Conditions** | User logged in; database contains reports. |
| **Post-Conditions** | Reports are visible and ready for interaction (details, voting, etc.). |
| **Exceptions** | **E1:** No reports found.<br>**E2:** Network failure.<br>**E3:** Database read error. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | View Reports List |
| **Use Case ID** | UC-U05 |
| **Description** | Displays all reports formatted in a scrolling list for easier browsing. Users can view timestamps, icons, locations, and summary data. |
| **Actions** | 1. User opens “Reports List”.<br>2. App fetches all report metadata from Firestore.<br>3. System loads list with titles, date/time, and thumbnails.<br>4. User scrolls and selects items. |
| **Alternative Paths** | **A1:** User sorts by newest, nearest, or severity.<br>**A2:** User refreshes the list. |
| **Pre-Conditions** | At least one report exists in the database. |
| **Post-Conditions** | List of reports displayed correctly. |
| **Exceptions** | **E1:** List cannot load due to network failure.<br>**E2:** Some report entries may have missing fields. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | View Reports on Map |
| **Use Case ID** | UC-U06 |
| **Description** | Shows disaster reports as geographic markers on an interactive map. Users can visualize the location of nearby incidents and tap on markers to view details. |
| **Actions** | 1. User opens “Map View”.<br>2. App initializes Google Maps API.<br>3. App loads all latitude/longitude data.<br>4. App places pins on the map.<br>5. User taps markers for details. |
| **Alternative Paths** | **A1:** User enables location to auto-center on their position.<br>**A2:** User filters map markers. |
| **Pre-Conditions** | Google Maps API enabled; internet active. |
| **Post-Conditions** | Map markers for all reports displayed. |
| **Exceptions** | **E1:** Map fails to load.<br>**E2:** Location permission denied.<br>**E3:** API quota exceeded. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | View Report Details |
| **Use Case ID** | UC-U07 |

| | |
|---|---|
| **Description** | Allows users to view full information about a specific disaster report including description, time, location, media proof, and voting activity. |
| **Actions** | 1. User selects a report.<br>2. System fetches full report data from Firestore.<br>3. Details page loads with media, map snapshot, and vote counts.<br>4. User may vote, view history, or flag content. |
| **Alternative Paths** | **A1:** User opens media in full screen.<br>**A2:** User views report on map from details page. |
| **Pre-Conditions** | Report must be available in the database. |
| **Post-Conditions** | User sees all relevant information for that report. |
| **Exceptions** | **E1:** Report deleted while user tries to access it.<br>**E2:** Media files unavailable. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | Create Disaster Report |
| **Use Case ID** | UC-U08 |
| **Description** | Allows users to submit a new incident report with optional media and auto-fetched location. Ensures accurate and real-time disaster reporting. |
| **Actions** | 1. User clicks “Create Report”.<br>2. User enters report details (type, description).<br>3. System auto-fetches user location.<br>4. User uploads/captures media.<br>5. User taps “Submit”.<br>6. Data & media uploaded to Firebase. |

| | |
|---|---|
| **Alternative Paths** | **A1:** User manually enters location if GPS fails.<br>**A2:** User submits report without media. |
| **Pre-Conditions** | User must be logged in and have required permissions. |
| **Post-Conditions** | New report becomes visible to the community. |
| **Exceptions** | **E1:** Location services disabled.<br>**E2:** Media upload failure.<br>**E3:** Mandatory fields left empty. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | Capture / Upload Media |
| **Use Case ID** | UC-U09 |
| **Description** | Allows users to attach photographic or video evidence to a new disaster report. The system accesses device cameras or gallery and uploads selected media to Firebase Storage. |
| **Actions** | 1. User taps “Add Media”.<br>2. User chooses Camera or Gallery.<br>3. User records or selects file.<br>4. App compresses and uploads to Firebase Storage.<br>5. Upload URL attached to report data. |
| **Alternative Paths** | **A1:** User skips media attachment.<br>**A2:** User retries upload after failure. |
| **Pre-Conditions** | Camera/gallery permissions granted. |
| **Post-Conditions** | Media stored and linked to report entry. |
| **Exceptions** | **E1:** Permission denied.<br>**E2:** File too large. |

| | |
|---|---|
| | **E3:** Upload interrupted due to lost connection. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | Auto-Fetch User Location |
| **Use Case ID** | UC-U10 |
| **Description** | System automatically retrieves the device's precise latitude and longitude to associate with a new disaster report. It enhances accuracy and reduces user effort. |
| **Actions** | 1. User opens the report form.<br>2. System requests GPS access if not granted.<br>3. GPS coordinates retrieved via Google Location API.<br>4. Coordinates inserted automatically into form. |
| **Alternative Paths** | **A1:** User denies location permission and enters manually.<br>**A2:** System fetches approximate location via network if GPS unavailable. |
| **Pre-Conditions** | GPS enabled and permission granted. |
| **Post-Conditions** | Location is attached to the report submission. |
| **Exceptions** | **E1:** GPS timeout.<br>**E2:** Location inaccurate.<br>**E3:** App cannot access location API. |
| **Author** | User |

| Field | Details |
|---|---|
| **Use Case Title** | Edit Own Report |
| **Use Case ID** | UC-U11 |
| **Description** | This use case allows a user to modify details of a disaster report they previously created. The user may update the description, attached media, or correct their location if it was inaccurate. The system ensures the user can only edit their own submissions. |
| **Actions / Main Flow** | 1. User opens “My Reports.”<br>2. User selects a report to edit.<br>3. System loads existing report data.<br>4. User updates fields such as description, media, or disaster type.<br>5. User saves changes.<br>6. System validates and updates the report in the database. |
| **Alternative Paths** | • User cancels editing → system discards changes.<br>• Media upload fails → system prompts user to retry or remove the media. |
| **Pre-conditions** | • User is authenticated.<br>• Report belongs to the user. |
| **Post-conditions** | • Updated report saved to the database.<br>• Users viewing the report receive updated information. |
| **Author** | User |
| **Exceptions** | • Database not reachable.<br>• User attempts to edit an admin-locked or resolved report. |

| Field | Details |
|---|---|
| **Use Case Title** | Delete Report |
| **Use Case ID** | UC-U12 |

| | |
|---|---|
| **Description** | Allows a user to permanently remove a report they created from the system. This is useful when the report was a mistake or no longer relevant. |
| **Actions / Main Flow** | 1. User opens “My Reports.”<br>2. User selects a report to delete.<br>3. User presses “Delete.”<br>4. System asks for confirmation.<br>5. User confirms deletion.<br>6. System removes the report and associated media. |
| **Alternative Paths** | • User cancels deletion → no changes made. |
| **Pre-conditions** | • User is authenticated.<br>• Report belongs to the user. |
| **Post-conditions** | • Report and media removed from database and storage. |
| **Author** | User |
| **Exceptions** | • Storage cleanup error.<br>• Deletion denied due to admin lock. |

| Field | Details |
|---|---|
| **Use Case Title** | Vote on Report |
| **Use Case ID** | UC-U13 |
| **Description** | Users contribute to the accuracy of reports by voting on their validity. Voting helps decide whether the report is real or false. |
| **Actions / Main Flow** | 1. User opens a report.<br>2. User taps “Vote.”<br>3. System displays voting options: Confirm or Dismiss.<br>4. User selects one. |
| **Alternative Paths** | • User already voted → system redirects to “Undo Vote.” |
| **Pre-conditions** | • User is authenticated. |

| | |
|---|---|
| **Post-conditions** | • System records user's vote. |
| **Author** | User |
| **Exceptions** | • Vote submission fails due to network issues. |

| Field | Details |
|---|---|
| **Use Case Title** | Cast Confirm Vote |
| **Use Case ID** | UC-U14 |
| **Description** | Allows the user to mark a report as trustworthy or real. These votes help the system validate the incident. |
| **Actions / Main Flow** | 1. User selects "Confirm Vote."<br>2. System records the vote in the database.<br>3. Vote count updates instantly. |
| **Alternative Paths** | • User tries to confirm a second time → system shows "Already voted." |
| **Pre-conditions** | • User is authenticated.<br>• User has not already voted. |
| **Post-conditions** | • Confirm vote stored. |
| **Author** | User |
| **Exceptions** | • Database write failure. |

| Field | Details |
|---|---|
| **Use Case Title** | Cast Dismiss Vote |
| **Use Case ID** | UC-U15 |
| **Description** | Allows the user to vote that the report is inaccurate or misleading. Helps flag false alarms quickly. |

| | |
|---|---|
| **Actions / Main Flow** | 1. User selects “Dismiss Vote.”<br>2. System saves the vote.<br>3. Vote counts update for all users. |
| **Alternative Paths** | • User tries to vote again → system restricts. |
| **Pre-conditions** | • User is authenticated.<br>• No previous vote exists. |
| **Post-conditions** | • Dismiss vote recorded. |
| **Author** | User |
| **Exceptions** | • Database write fails. |

| Field | Details |
|---|---|
| **Use Case Title** | Undo Vote |
| **Use Case ID** | UC-U16 |
| **Description** | Enables a user to remove their previous vote from a report, giving them the flexibility to change their decision. |
| **Actions / Main Flow** | 1. User opens a report they voted on.<br>2. User taps “Undo Vote.”<br>3. System removes the vote from the database. |
| **Alternative Paths** | • User has not voted → system hides the undo button. |
| **Pre-conditions** | • User is authenticated.<br>• User has a vote on that report. |
| **Post-conditions** | • Vote removed; counts updated. |
| **Author** | User |
| **Exceptions** | • Database error while removing vote. |

| Field | Details |
|---|---|
| **Use Case Title** | View Vote Counts / History |
| **Use Case ID** | UC-U17 |
| **Description** | Allows users to see how many people confirmed or dismissed the report, and optionally view a timeline of vote changes. This increases transparency. |
| **Actions / Main Flow** | 1. User opens report details.<br>2. System loads vote statistics.<br>3. User views counts or expands history. |
| **Alternative Paths** | • No votes exist → system shows “No votes yet.” |
| **Pre-conditions** | • User is authenticated. |
| **Post-conditions** | • User understands the reliability of the report. |
| **Author** | User |
| **Exceptions** | • Stats fail to load due to network issues. |

| Field | Details |
|---|---|
| **Use Case Title** | Report Abuse / Flag Content |
| **Use Case ID** | UC-U18 |
| **Description** | Users can flag reports or comments they believe contain false information, inappropriate content, or harmful media. These flags help admins maintain the platform’s safety. |
| **Actions / Main Flow** | 1. User opens a report.<br>2. User taps “Flag Report.”<br>3. User selects a reason (misleading, inappropriate media, spam, etc.).<br>4. System submits the flag to the admin dashboard. |
| **Alternative Paths** | • User cancels the flag. |

| | |
|---|---|
| **Pre-conditions** | • User is authenticated. |
| **Post-conditions** | • Admin notified of flagged content. |
| **Author** | User |
| **Exceptions** | • Network failure prevents flag submission. |

| Field | Details |
|---|---|
| **Use Case Title** | Receive Notifications |
| **Use Case ID** | UC-U19 |
| **Description** | Users receive push notifications for nearby disasters, updates to their reports, votes, admin verification, SOS alerts, and app announcements. |
| **Actions / Main Flow** | 1. System checks user location and preferences.<br>2. A new event triggers a notification.<br>3. Firebase Cloud Messaging sends alert to user's device.<br>4. User sees the notification. |
| **Alternative Paths** | • User turned off notifications → system silently ignores. |
| **Pre-conditions** | • App permissions granted.<br>• FCM configured. |
| **Post-conditions** | • User stays informed in real-time. |
| **Author** | User |
| **Exceptions** | • Notification delivery failure. |

| Field | Details |
|---|---|
| **Use Case Title** | View Notification Details |
| **Use Case ID** | UC-U20 |
| **Description** | Allows a user to open a received notification and view the full details, such as report information, SOS alert details, or admin updates. |
| **Actions / Main Flow** | 1. User taps on notification.<br>2. App launches the relevant screen (report detail, SOS event, etc.).<br>3. System loads the full content. |
| **Alternative Paths** | • Notification expired → system displays a fallback message. |
| **Pre-conditions** | • Notification exists on device. |
| **Post-conditions** | • User reads detailed information. |
| **Author** | User |
| **Exceptions** | • Linked report deleted by admin. |

| Field | Details |
|---|---|
| **Use Case Title** | Send SOS Alert |
| **Use Case ID** | UC-U21 |
| **Description** | The user triggers an emergency SOS alert when they are in danger. The app sends their live location, timestamp, and an alert message to all saved emergency contacts and stores the SOS event in the database for admin visibility. |
| **Actions / Main Flow** | 1. User taps the SOS button.<br>2. System asks for confirmation (to prevent accidental triggers).<br>3. User confirms SOS.<br>4. App fetches user's live location.<br>5. System sends alert to all saved contacts (SMS or in-app if supported). |

| | |
|---|---|
| | 6. System logs event in Firebase. |
| **Alternative Paths** | • User cancels SOS at confirmation.<br>• Location not available → system uses last known location. |
| **Pre-conditions** | • SOS contacts added.<br>• Location permission granted. |
| **Post-conditions** | • Emergency contacts receive alert.<br>• SOS event stored in database. |
| **Author** | User |
| **Exceptions** | • No internet.<br>• Contacts unreachable.<br>• GPS not working. |

| Field | Details |
|---|---|
| **Use Case Title** | Send Location to Contacts |
| **Use Case ID** | UC-U22 |
| **Description** | Sends the user's real-time or latest available GPS location directly to all registered emergency contacts, usually triggered by an SOS event. |
| **Actions / Main Flow** | 1. App retrieves the user’s current coordinates.<br>2. System formats a location-sharing message.<br>3. Message sent to all saved contacts (SMS / push). |
| **Alternative Paths** | • Location accuracy poor → system indicates approximate location.<br>• User cancels sharing. |
| **Pre-conditions** | • User is authenticated.<br>• Location access enabled. |
| **Post-conditions** | • Contacts receive the location message. |
| **Author** | User |
| **Exceptions** | • SMS sending failure.<br>• GPS fetch error. |

| Field | Details |
| --- | --- |
| **Use Case Title** | Store SOS Event |
| **Use Case ID** | UC-U23 |
| **Description** | The app logs the SOS alert details (user ID, timestamp, location, contact status) in Firebase so that admins can track emergency cases and verify patterns. |
| **Actions / Main Flow** | 1. SOS triggered.2. System creates a new SOS entry.3. Stores location, time, device ID, and status.4. Admin panel shows the SOS alert. |
| **Alternative Paths** | • System retries saving when offline. |
| **Pre-conditions** | • SOS initiated by user. |
| **Post-conditions** | • SOS event stored successfully. |
| **Author** | User |
| **Exceptions** | • Firebase write fails.<br>• Duplicate event entries created. |

| Field | Details |
| --- | --- |
| **Use Case Title** | Manage SOS Contacts |
| **Use Case ID** | UC-U24 |
| **Description** | Allows users to add, edit, or remove emergency contacts who will receive SOS alerts. |

| | |
|---|---|
| **Actions / Main Flow** | 1. User opens “SOS Contacts.”<br>2. User adds a new contact (name + phone number).<br>3. System validates format.<br>4. User can also update or remove existing contacts.<br>5. System saves contact list to the database. |
| **Alternative Paths** | • User enters invalid phone → system shows error.<br>• User cancels action. |
| **Pre-conditions** | • User authenticated. |
| **Post-conditions** | • Updated list stored and used for future SOS alerts. |
| **Author** | User |
| **Exceptions** | • Contacts limit exceeded.<br>• Database unavailable. |

| Field | Details |
|---|---|
| **Use Case Title** | Admin Login |
| **Use Case ID** | UC-A25 |
| **Description** | Admin logs into a secure admin panel where they can manage user reports, media, and system notifications. Admin authentication is handled through Firebase Authentication. |
| **Actions / Main Flow** | 1. Admin opens login portal.<br>2. Enters email and password.<br>3. System validates credentials.<br>4. Admin dashboard loads. |
| **Alternative Paths** | • Wrong password → system rejects and shows error. |
| **Pre-conditions** | • Admin account already exists. |

| | |
|---|---|
| **Post-conditions** | • Admin gains access to dashboard. |
| **Author** | Admin |
| **Exceptions** | • Admin account suspended.<br>• Network or Firebase failure. |

| Field | Details |
|---|---|
| **Use Case Title** | View All Reports |
| **Use Case ID** | UC-A26 |
| **Description** | Allows admins to view all disaster reports submitted in the system, including resolved and flagged ones. |
| **Actions / Main Flow** | 1. Admin opens “Reports” section.<br>2. System loads a list of all reports.<br>3. Admin filters by date, type, status, or flags if needed. |
| **Alternative Paths** | • No reports available → system displays empty state. |
| **Pre-conditions** | • Admin is logged in. |
| **Post-conditions** | • Admin sees complete report statistics. |
| **Author** | Admin |
| **Exceptions** | • Database load failure. |

| Field | Details |
|---|---|
| **Use Case Title** | Review Report Details |
| **Use Case ID** | UC-A27 |

| | |
|---|---|
| **Description** | Admin inspects a specific report, including its media, votes, user info, and flags, to decide how to handle it. |
| **Actions / Main Flow** | 1. Admin selects a report.<br>2. System displays full details, including media and vote stats.<br>3. Admin reviews data. |
| **Alternative Paths** | • Report deleted by user → system shows “Report not found.” |
| **Pre-conditions** | • Admin authenticated. |
| **Post-conditions** | • Admin prepared to verify, resolve, or dismiss. |
| **Author** | Admin |
| **Exceptions** | • Media fails to load. |

| Field | Details |
|---|---|
| **Use Case Title** | Review Report Details |
| **Use Case ID** | UC-A27 |
| **Description** | Admin inspects a specific report, including its media, votes, user info, and flags, to decide how to handle it. |
| **Actions / Main Flow** | 1. Admin selects a report.<br>2. System displays full details, including media and vote stats.<br>3. Admin reviews data. |
| **Alternative Paths** | • Report deleted by user → system shows “Report not found.” |
| **Pre-conditions** | • Admin authenticated. |
| **Post-conditions** | • Admin prepared to verify, resolve, or dismiss. |
| **Author** | Admin |
| **Exceptions** | • Media fails to load. |

| Field | Details |
|---|---|
| **Use Case Title** | Verify Report |
| **Use Case ID** | UC-A28 |
| **Description** | Admin formally confirms that a report is legitimate, based on evidence or community votes. |
| **Actions / Main Flow** | 1. Admin opens a report.<br>2. Admin taps “Verify Report.”<br>3. System updates the status to “Verified.”<br>4. Notifications are sent to the report creator. |
| **Alternative Paths** | • Admin changes mind → does nothing. |
| **Pre-conditions** | • Report exists. |
| **Post-conditions** | • Verified status displayed to all users. |
| **Author** | Admin |
| **Exceptions** | • Database update fails. |

| Field | Details |
|---|---|
| **Use Case Title** | Mark Report as Resolved |
| **Use Case ID** | UC-A29 |
| **Description** | Admin marks a previously active disaster report as “Resolved” when the issue has been handled or is no longer active. |
| **Actions / Main Flow** | 1. Admin opens active report.<br>2. Taps “Mark as Resolved.”<br>3. System updates status everywhere. |

| | |
|---|---|
| **Alternative Paths** | • Admin reopens report instead of resolving. |
| **Pre-conditions** | • Report is active. |
| **Post-conditions** | • Status changes to “Resolved.” |
| **Author** | Admin |
| **Exceptions** | • Update fails due to network/server issues. |

| Field | Details |
|---|---|
| **Use Case Title** | Mark Report as False Alarm |
| **Use Case ID** | UC-A30 |
| **Description** | Admin marks a report as false when multiple dismiss votes, misleading media, or user abuse flags indicate the report is not real. |
| **Actions / Main Flow** | 1. Admin opens the report.<br>2. Selects “False Alarm.”<br>3. System updates status.<br>4. Reporter receives a notification. |
| **Alternative Paths** | • Admin chooses to delete the report entirely. |
| **Pre-conditions** | • Report exists. |
| **Post-conditions** | • False alarm label displayed system-wide. |
| **Author** | Admin |
| **Exceptions** | • Database update fails.<br>• Admin privileges not granted. |

| Field | Details |
|---|---|
| **Use Case Title** | Delete Inappropriate Media |
| **Use Case ID** | UC-A31 |
| **Description** | Admin can remove media (images/videos) attached to reports if it is harmful, irrelevant, or violates policies. |
| **Actions / Main Flow** | 1. Admin opens a report with media.<br>2. Selects “Delete Media.”<br>3. System asks for confirmation.<br>4. Media is removed from Firebase Storage and report updated. |
| **Alternative Paths** | • Admin cancels deletion → no changes made. |
| **Pre-conditions** | • Admin authenticated.<br>• Media exists in report. |
| **Post-conditions** | • Media removed and report updated. |
| **Author** | Admin |
| **Exceptions** | • Firebase Storage deletion fails. |

| Field | Details |
|---|---|
| **Use Case Title** | Manage Users |
| **Use Case ID** | UC-A32 |
| **Description** | Admin can view, update, suspend, or delete user accounts to maintain platform integrity. |
| **Actions / Main Flow** | 1. Admin opens “Users” panel.<br>2. System displays all users.<br>3. Admin selects a user and performs actions: Update, Suspend, Delete.<br>4. System updates database and notifies affected user if applicable. |

| | |
|---|---|
| **Alternative Paths** | • Admin cancels any operation → no changes made. |
| **Pre-conditions** | • Admin authenticated. |
| **Post-conditions** | • User status updated as per action. |
| **Author** | Admin |
| **Exceptions** | • Database update fails.<br>• Admin lacks privileges. |

| Field | Details |
|---|---|
| **Use Case Title** | Update User |
| **Use Case ID** | UC-A33 |
| **Description** | Admin edits user details such as name, email, or role. |
| **Actions / Main Flow** | 1. Admin selects a user.<br>2. Updates details in the form.<br>3. System validates inputs.<br>4. Changes saved in database. |
| **Alternative Paths** | • Invalid data → system shows error.<br>• Admin cancels update. |
| **Pre-conditions** | • User exists.<br>• Admin authenticated. |
| **Post-conditions** | • User details updated. |
| **Author** | Admin |
| **Exceptions** | • Database connection failure. |

| Field | Details |
|---|---|
| **Use Case Title** | Suspend User |
| **Use Case ID** | UC-A34 |
| **Description** | Temporarily disables a user's account for violating rules or suspicious activity. |
| **Primary Actor** | Admin |
| **Actions / Main Flow** | 1. Admin selects user.<br>2. Chooses "Suspend."<br>3. System disables login access and logs reason.<br>4. User notified via email. |
| **Alternative Paths** | • Admin cancels suspension → no changes made. |
| **Pre-conditions** | • User exists.<br>• Admin authenticated. |
| **Post-conditions** | • User cannot log in until reinstated. |
| **Author** | Admin |
| **Exceptions** | • Notification failure.<br>• Database error. |

| Field | Details |
|---|---|
| **Use Case Title** | Delete User |
| **Use Case ID** | UC-A35 |
| **Description** | Admin permanently removes a user account along with their reports, votes, and media if necessary. |
| **Actions / Main Flow** | 1. Admin selects user.<br>2. Chooses "Delete User."<br>3. System asks for confirmation.<br>4. System deletes user data and removes access. |

| | |
|---|---|
| **Alternative Paths** | • Admin cancels → no deletion occurs. |
| **Pre-conditions** | • Admin authenticated.<br>• User exists. |
| **Post-conditions** | • User removed from system. |
| **Author** | Admin |
| **Exceptions** | • Database deletion error.<br>• Media cleanup fails. |

| Field | Details |
|---|---|
| **Use Case Title** | Manage Notifications |
| **Use Case ID** | UC-A36 |
| **Description** | Admin can send, update, or delete notifications to users. Notifications may include alerts, system messages, or announcements. |
| **Actions / Main Flow** | 1. Admin opens notifications panel.<br>2. Creates or selects existing notification.<br>3. Updates content, audience, or deletes it.<br>4. System pushes updates via Firebase Cloud Messaging. |
| **Alternative Paths** | • Admin cancels → no changes made. |
| **Pre-conditions** | • Admin authenticated. |
| **Post-conditions** | • Notifications sent or updated. |
| **Author** | Admin |
| **Exceptions** | • FCM service failure.<br>• Database update error. |

| Field | Details |
|---|---|
| **Use Case Title** | Update Notification |
| **Use Case ID** | UC-A37 |
| **Description** | Admin edits an existing notification before sending it to ensure accurate information. |
| **Actions / Main Flow** | 1. Admin selects notification.<br>2. Updates content or audience.<br>3. Saves changes.<br>4. System pushes updated notification. |
| **Alternative Paths** | • Admin cancels edit → no changes made. |
| **Pre-conditions** | • Notification exists.<br>• Admin authenticated. |
| **Post-conditions** | • Notification updated and sent. |
| **Author** | Admin |
| **Exceptions** | • FCM failure.<br>• Database update failure. |

| Field | Details |
|---|---|
| **Use Case Title** | Delete Notification |
| **Use Case ID** | UC-A38 |
| **Description** | Admin deletes outdated or incorrect notifications from the system. |
| **Actions / Main Flow** | 1. Admin selects a notification.<br>2. Confirms deletion.<br>3. System removes notification from database and prevents delivery. |
| **Alternative Paths** | • Admin cancels → no deletion occurs. |
| **Pre-conditions** | • Notification exists. |

| | |
|---|---|
| | • Admin authenticated. |
| **Post-conditions** | • Notification removed. |
| **Author** | Admin |
| **Exceptions** | • Database deletion failure. |

| Field | Details |
|---|---|
| **Use Case Title** | Store Media Files |
| **Use Case ID** | UC-S39 |
| **Description** | All media (images/videos) attached to reports or SOS events are uploaded and stored securely in Firebase Storage. |
| **Actions / Main Flow** | 1. User attaches media.<br>2. System uploads file to Firebase.<br>3. Generates secure URL.<br>4. URL saved in Firestore with report/event data. |
| **Alternative Paths** | • Upload fails → system prompts retry.<br>• User cancels upload. |
| **Pre-conditions** | • Firebase Storage configured.<br>• User authenticated. |
| **Post-conditions** | • Media securely stored.<br>• Accessible for report display. |
| **Author** | System |
| **Exceptions** | • Upload interrupted.<br>• File size too large. |

| Field | Details |
|---|---|
| **Use Case Title** | Send Push Notification |

| | |
|---|---|
| **Use Case ID** | UC-S40 |
| **Description** | System sends real-time push notifications to users for reports, updates, SOS events, or admin alerts using Firebase Cloud Messaging. |
| **Primary Actor** | System / Firebase |
| **Actions / Main Flow** | 1. Event occurs (new report, SOS, admin update).<br>2. System prepares notification payload.<br>3. Sends via Firebase Cloud Messaging.<br>4. Users receive notification on device. |
| **Alternative Paths** | • User disabled notifications → message not delivered. |
| **Pre-conditions** | • User has app installed.<br>• FCM configured. |
| **Post-conditions** | • Users receive real-time notifications. |
| **Author** | System |
| **Exceptions** | • FCM service down.<br>• Network failure. |

Adopted Methodology

The **VU Process Model** is a hybrid software development methodology that combines the **Waterfall** and **Spiral** models. It provides a structured, sequential approach while incorporating iterative refinement and risk management, which is particularly useful for projects that involve user interaction, real-time data, and cloud integration.

**1. Requirements Analysis**

The project begins with a structured collection of requirements from stakeholders, including potential app users, emergency responders, and authorities. This phase ensures that all functional and non-functional requirements are clearly understood and documented.

**2. System Design**

Using the requirements, the system design phase develops a detailed architecture for the app. Key design aspects include:

- Defining the data flow between users, Firebase, and the admin panel
- Designing the user interface for reports, map views, and SOS alerts
- Structuring database schemas and storage strategies
- Planning push notifications and real-time updates

**3. Prototyping and Risk Analysis**

In this iterative Spiral phase, prototypes of critical components are developed and tested. High-risk areas, such as:

- Real-time location tracking
- Push notifications reliability
- Verification system for disaster reports are addressed first.

Prototypes are shared with test users to gather early feedback, which is used to refine the system and mitigate potential risks before full-scale development.

**4. Development**

With the design and prototypes validated, the main development phase begins. Features are implemented in a structured, step-by-step manner while integrating the feedback from the prototyping phase. This ensures that core functionalities like disaster reporting, map visualization, SOS alerts, and admin control are developed in alignment with the planned architecture.

**5. Testing and Verification**

Testing is carried out simultaneously with development, following both unit and system-level testing approaches. Key testing activities include:

- Unit testing for individual features (report creation, notifications, SOS alerts)
- Integration testing for interactions between modules (Firebase integration, map updates)
- System testing to validate that the app meets all functional requirements

### 6. Deployment

After successful testing, the app is deployed on the Android platform via APK or the Google Play Store. Deployment ensures that users can access the app in real-time, enabling immediate community reporting and disaster alerts.

### 7. Maintenance and Continuous Improvement

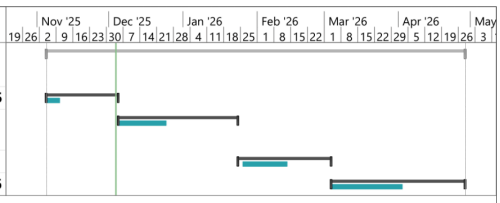
Post-deployment, the app enters the maintenance phase. Real-world feedback is continuously analyzed, and improvements are made, including:

- Bug fixes and performance enhancements
- Adding new disaster categories or features
- Refining push notification and SOS functionalities

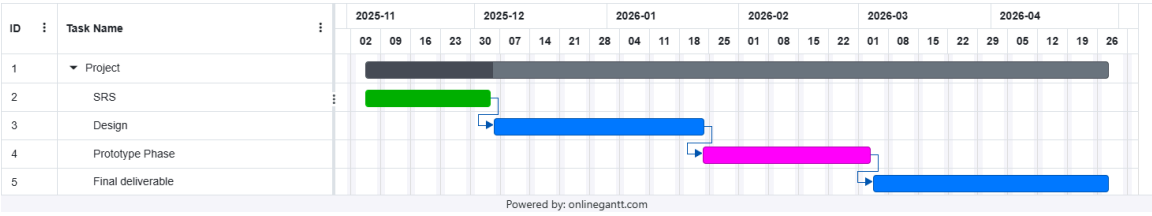
The **VU Process Model** provides a balanced development approach for the **Crowdsourced Disaster Alert & Safety App**, combining the systematic, stepwise planning of the Waterfall model with the flexibility, risk management, and iterative improvements of the Spiral model. This ensures a reliable, user-centered application capable of providing timely disaster alerts and enhancing public safety.

# Work Plan (Use MS Project to create Schedule/Work Plan)

| ID | | Task Mode | Task Name | Duration | Start | Finish |
|---|---|---|---|---|---|---|
| 0 | | | **Software Development** | **125 days** | **Wed 11/5/25** | **Tue 4/28/26** |
| 1 | | | **SRS** | **22 days** | **Wed 11/5/25** | **Thu 12/4/25** |
| 7 | | | **Design** | **36 days** | **Fri 12/5/25** | **Fri 1/23/26** |
| 17 | | | **Prototype Phase** | **28 days** | **Sat 1/24/26** | **Tue 3/3/26** |
| 25 | | | **Final Deliverable** | **40 days** | **Wed 3/4/26** | **Tue 4/28/26** |




Project: Software Development
Date: Thu 12/4/25

| | | |
|---|---|---|
| Task | Inactive Summary | External Tasks |
| Split | Manual Task | External Milestone |
| Milestone | Duration-only | Deadline |
| Summary | Manual Summary Rollup | Progress |
| Project Summary | Manual Summary | Manual Progress |
| Inactive Task | Start-only | |
| Inactive Milestone | Finish-only | |

Page 1